sequence of events is shown in figure 2.8, and is as follows:

1. When the form is submitted, the browser sends a POST request to http://localhost:4567/spaces with a Content-Type header of text/plain and the hidden form field as the value. When the browser submits the form, it takes each form element and submits them as name=value pairs. The `<`, `>` and `'` HTML entities are replaced with the literal values <, >, and ’ respectively.
2. The name of your hidden input field is ‘`{"x":"`’, although the value is your long malicious script. When the two are put together the API will see the following form input:
   ```
   {"x":"=","name":"x","owner":"<script>alert('XSS!');
   </script>"}
   ```
3. The API sees a valid JSON input and ignores the extra “x” field (which you only added to cleverly hide the equals sign that the browser will add). But the API rejects the username as invalid, echoing it back in the response:
   ```
   {"error": "java.lang.IllegalArgumentException: invalid
   username: <script>alert('XSS!');</script>"}
   ```
4. Because your error response was served with the default Content-Type of `text/html`, the browser happily interprets the response as HTML and executes the script, resulting in the XSS popup.